Ψηφιακά Συστήματα VLSI

2η Εργαστηριακή Άσκηση

Ομάδα 32: Νικόλαος Λοβέρδος (03120442)

Δημήτριος Μιχαλακόπουλος (03120229)

Βάρδια Γ: 11:30-12:30

Σχολή: Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Ζητούμενο 1: Half Adder - HA

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity half_adder is
5       Port ( A, B : in  std_logic;
6              Sum, Carry : out  std_logic);
7   end half_adder;
8
9   architecture dataflow of half_adder is
10  begin
11      Sum <= A xor B;
12      Carry <= A and B;
13  end dataflow;
```

Ο πίνακας αληθείας του κυκλώματος του ημιαθροιστή είναι ο εξής:

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Για το άθροισμα και το κρατούμενο προκύπτουν οι παρακάτω λογικές εκφράσεις:

Sum = A'B OR AB' = A XOR B

Carry = A AND B

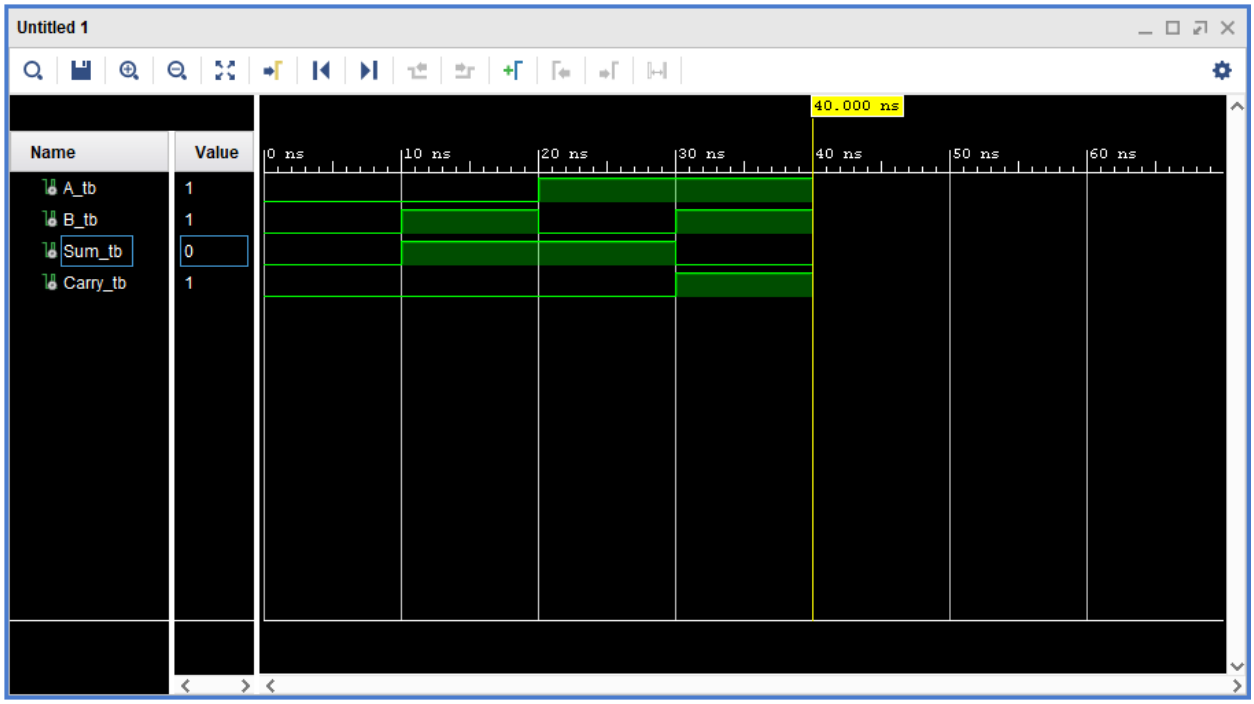
Ο παραπάνω κώδικας, λοιπόν, υλοποιεί έναν ημιαθροιστή που λαμβάνει δύο bit εισόδου A και B, και παράγει δύο bit εξόδου, το Sum, το οποίο προκύπτει από την λογική πράξη XOR των εισόδων, και το Carry, το οποίο προκύπτει από την λογική πράξη AND των εισόδων.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity half_adder_tb is
5   end half_adder_tb;
6
7   architecture tb_arch of half_adder_tb is
8
9       component half_adder
10          Port ( A, B : in  std_logic;
11                 Sum, Carry : out  std_logic);
12      end component;
13
14      signal A_tb, B_tb, Sum_tb, Carry_tb : std_logic;
15
16  begin
17
18      DUT: half_adder port map (
19          A => A_tb,
20          B => B_tb,
21          Sum => Sum_tb,
22          Carry => Carry_tb
23      );
24
25      stimulus_proc: process
26      begin
27          A_tb <= '0';
28          B_tb <= '0';
29          wait for 10 ns;
30
31          A_tb <= '0';
32          B_tb <= '1';
33          wait for 10 ns;
34
35          A_tb <= '1';
36          B_tb <= '0';
37          wait for 10 ns;
38
39          A_tb <= '1';
40          B_tb <= '1';
41          wait for 10 ns;
42
43          wait;
44      end process;
45  end tb_arch;
```

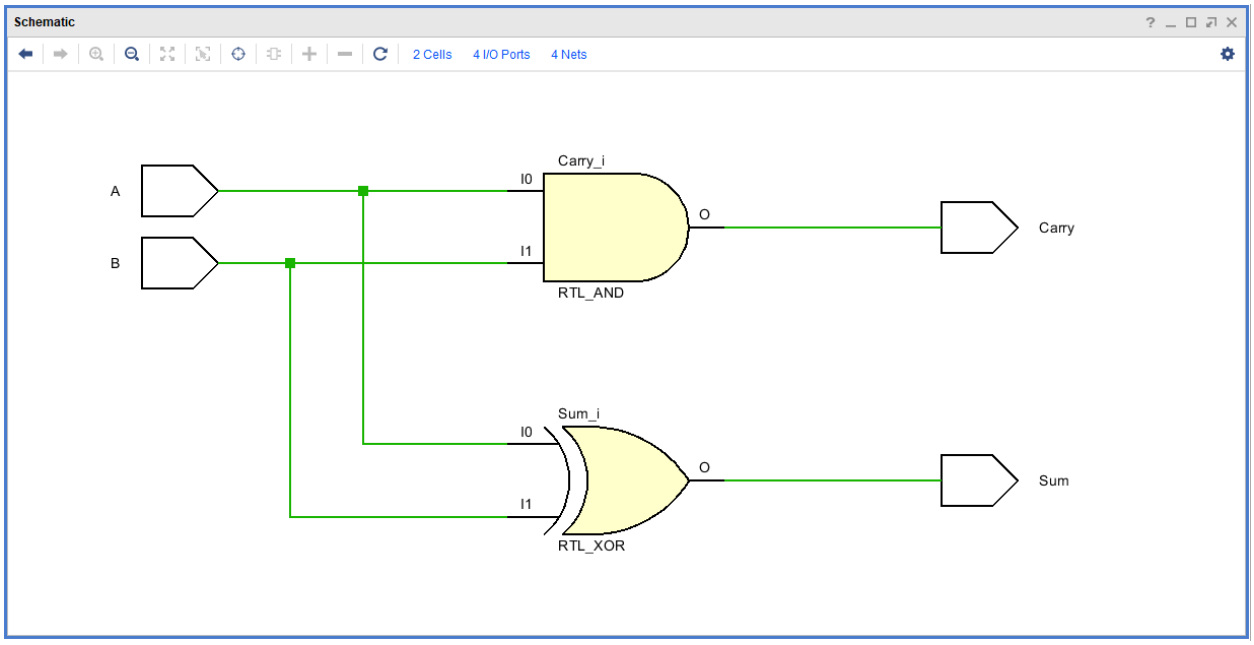
Το παραπάνω testbench εφαρμόζει στον ημιαθροιστή όλους τους διαφορετικούς συνδυασμούς εισόδων προκειμένου να εξετάσουμε τις αντίστοιχες εξόδους του.

Η προσομοίωση του κυκλώματος του Half Adder φαίνεται παρακάτω:



Παρατηρούμε ότι για κάθε τιμή των εισόδων A και B του ημιαθροιστή, οι έξοδοι Sum και Carry παίρνουν κάθε φορά την αντίστοιχη τιμή που υπάρχει στον πίνακα αληθείας, επομένως το κύκλωμα του ημιαθροιστή λειτουργεί ορθά.

Το RTL του κυκλώματος του Half Adder φαίνεται παρακάτω:



Το κρίσιμο μονοπάτι του Half Adder ξεκινάει από την είσοδο B και καταλήγει στην έξοδο Sum με συνολική καθυστέρηση 5.377 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	3	4	2	B	Sum	5.377	3.778	1.599
Path 2	∞	3	4	2	B	Carry	5.351	3.752	1.599

Ζητούμενο 2: Full Adder - FA

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity full_adder is
5       Port ( A, B, Cin : in  std_logic;
6              Sum, Cout : out  std_logic);
7   end full_adder;
8
9   architecture structural of full_adder is
10
11      component half_adder
12          Port ( A, B : in  std_logic;
13                 Sum, Carry : out  std_logic);
14      end component;
15
16      signal Sum1, Carry1, Carry2 : std_logic;
17
18  begin
19
20      HA1: half_adder port map (
21          A => A,
22          B => B,
23          Sum => Sum1,
24          Carry => Carry1
25      );
26
27      HA2: half_adder port map (
28          A => Sum1,
29          B => Cin,
30          Sum => Sum,
31          Carry => Carry2
32      );
33
34      Cout <= Carry1 or Carry2;
35
36  end structural;
```

Ο πίνακας αληθείας του κυκλώματος του πλήρους αθροιστή είναι ο εξής:

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Ο παραπάνω κώδικας, λοιπόν, υλοποιεί έναν πλήρη αθροιστή με Structural αρχιτεκτονική χρησιμοποιώντας δύο Half Adder ως components. Ο πρώτος Half Adder πραγματοποιεί την πρόσθεση των εισόδων A και B, ενώ ο δεύτερος Half Adder παίρνει το προηγούμενο άθροισμα και του προσθέτει το κρατούμενο εισόδου Cin, με αποτέλεσμα να προκύπτει η έξοδος Sum του Full Adder. Τέλος, το κρατούμενο εξόδου Cout προκύπτει από την λογική πράξη OR των κρατούμενων εξόδου Carry των δύο Half Adder.

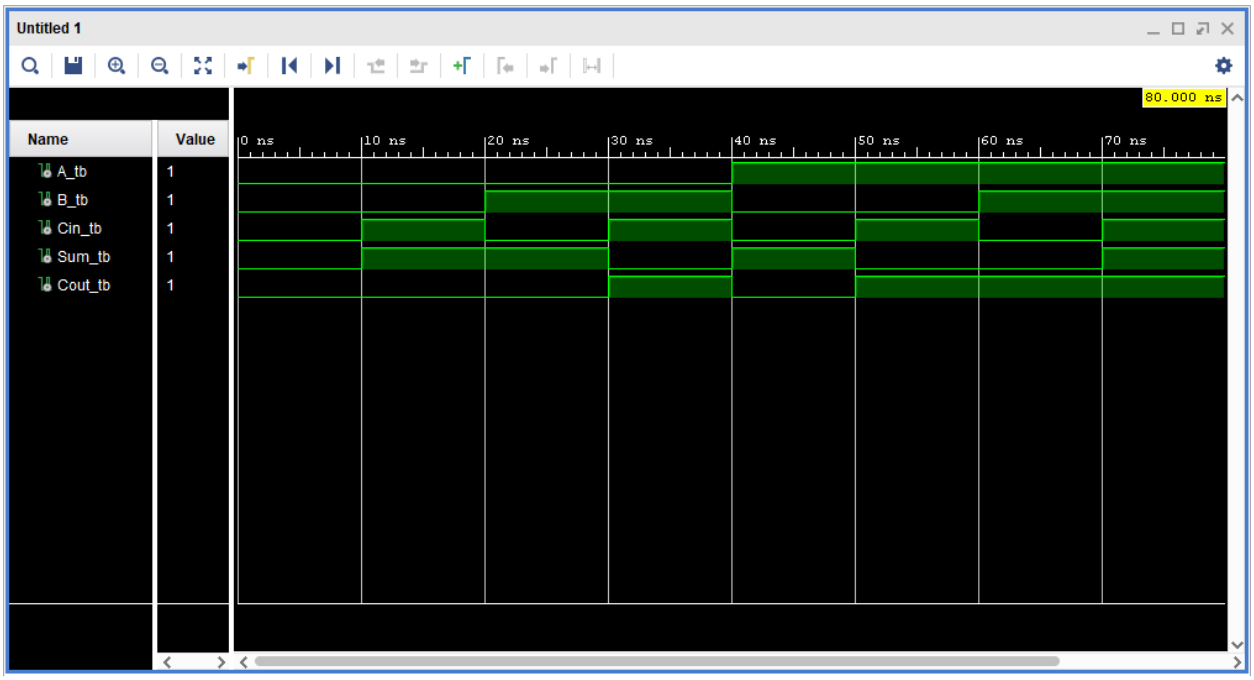
Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity full_adder_tb is
5   end full_adder_tb;
6
7   architecture tb_arch of full_adder_tb is
8
9       component full_adder
10          Port ( A, B, Cin : in  std_logic;
11                 Sum, Cout : out  std_logic);
12      end component;
13
14      signal A_tb, B_tb, Cin_tb, Sum_tb, Cout_tb : std_logic;
15
16  begin
17
18      DUT: full_adder port map (
19          A => A_tb,
20          B => B_tb,
21          Cin => Cin_tb,
22          Sum => Sum_tb,
23          Cout => Cout_tb
24      );
25
```

```
26      stimulus_proc: process
27      begin
28          A_tb <= '0';
29          B_tb <= '0';
30          Cin_tb <= '0';
31          wait for 10 ns;
32
33          A_tb <= '0';
34          B_tb <= '0';
35          Cin_tb <= '1';
36          wait for 10 ns;
37
38          A_tb <= '0';
39          B_tb <= '1';
40          Cin_tb <= '0';
41          wait for 10 ns;
42
43          A_tb <= '0';
44          B_tb <= '1';
45          Cin_tb <= '1';
46          wait for 10 ns;
47
48          A_tb <= '1';
49          B_tb <= '0';
50          Cin_tb <= '0';
51          wait for 10 ns;
52
53          A_tb <= '1';
54          B_tb <= '0';
55          Cin_tb <= '1';
56          wait for 10 ns;
57
58          A_tb <= '1';
59          B_tb <= '1';
60          Cin_tb <= '0';
61          wait for 10 ns;
62
63          A_tb <= '1';
64          B_tb <= '1';
65          Cin_tb <= '1';
66          wait for 10 ns;
67
68          wait;
69      end process;
70  end tb_arch;
```

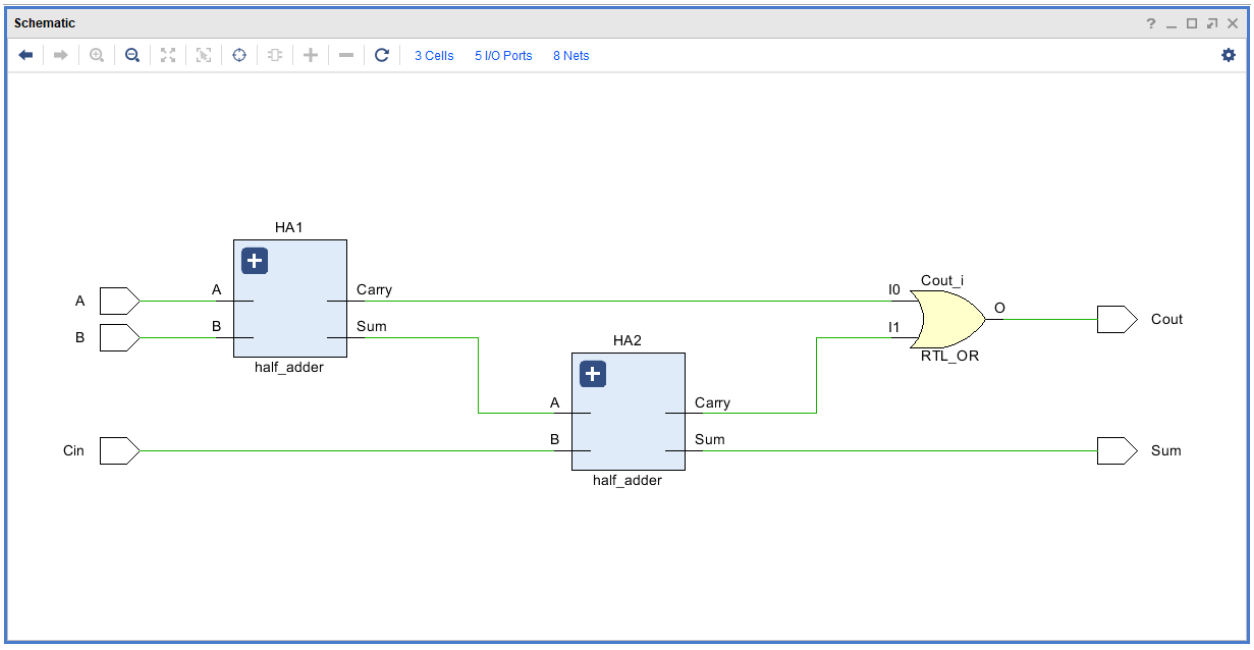
Το παραπάνω testbench εφαρμόζει όλους τους διαφορετικούς συνδυασμούς εισόδων στον πλήρη αθροιστή προκειμένου να εξετάσουμε τις αντίστοιχες εξόδους του.

Η προσομοίωση του κυκλώματος του Full Adder φαίνεται παρακάτω:



Παρατηρούμε ότι για κάθε τιμή των εισόδων A, B και Cin του πλήρους αθροιστή, οι έξοδοι Sum και Cout παίρνουν κάθε φορά την αντίστοιχη τιμή που υπάρχει στον πίνακα αληθείας, άρα το κύκλωμα του πλήρους αθροιστή λειτουργεί ορθά.

Το RTL του κυκλώματος του Full Adder φαίνεται παρακάτω:



Το κρίσιμο μονοπάτι του Full Adder ξεκινάει από την είσοδο A και καταλήγει στην έξοδο Sum με συνολική καθυστέρηση 5.377 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	3	4	2	A	Sum	5.377	3.778	1.599
Path 2	∞	3	4	2	A	Cout	5.351	3.752	1.599

Ζητούμενο 3: 4-bit Parallel Adder – 4-bit PA

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity four_bit_adder is
5       Port ( A, B : in  std_logic_vector(3 downto 0);
6              Cin    : in  std_logic;
7              Sum    : out std_logic_vector(3 downto 0);
8              Cout   : out std_logic);
9   end four_bit_adder;
10
11  architecture structural of four_bit_adder is
12
13      component full_adder
14          Port ( A, B, Cin : in  std_logic;
15                 Sum, Cout : out  std_logic);
16      end component;
17
18      signal Carry : std_logic_vector(3 downto 0);
19
20  begin
21
22      FA0: full_adder port map (
23          A => A(0),
24          B => B(0),
25          Cin => Cin,
26          Sum => Sum(0),
27          Cout => Carry(0)
28      );
```

```
29
30      FA1: full_adder port map (
31          A => A(1),
32          B => B(1),
33          Cin => Carry(0),
34          Sum => Sum(1),
35          Cout => Carry(1)
36      );
37
38      FA2: full_adder port map (
39          A => A(2),
40          B => B(2),
41          Cin => Carry(1),
42          Sum => Sum(2),
43          Cout => Carry(2)
44      );
45
46      FA3: full_adder port map (
47          A => A(3),
48          B => B(3),
49          Cin => Carry(2),
50          Sum => Sum(3),
51          Cout => Cout
52      );
53
54  end structural;
```

Ο παραπάνω κώδικας υλοποιεί έναν παράλληλο αθροιστή 4 bits με Structural αρχιτεκτονική χρησιμοποιώντας τέσσερις Full Adder ως components. Κάθε ένας από τους αθροιστές χειρίζεται ένα από τα τέσσερα bit της εξόδου Sum και με συνδεσμολογία cascade, όπου το κρατούμενο εξόδου ενός αθροιστή συνδέεται με τον κρατούμενο εισόδου του επόμενου, γίνεται η διάδοση του κρατούμενου. Προφανώς στον Full Adder που χειρίζεται το LSB γίνεται και πρόσθεση του κρατούμενου εισόδου Cin του παράλληλου αθροιστή και, αντίστοιχα, ο Full Adder που χειρίζεται το MSB δίνει το κρατούμενο εξόδου Cout.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity four_bit_adder_tb is
end four_bit_adder_tb;

architecture tb_arch of four_bit_adder_tb is

    component four_bit_adder
        Port ( A, B : in  std_logic_vector(3 downto 0);
               Cin    : in  std_logic;
               Sum    : out std_logic_vector(3 downto 0);
               Cout   : out std_logic);
    end component;

    signal A_tb, B_tb, Sum_tb : std_logic_vector(3 downto 0);
    signal Cin_tb, Cout_tb : std_logic;

begin

    DUT: four_bit_adder port map (
        A    => A_tb,
        B    => B_tb,
        Cin  => Cin_tb,
        Sum  => Sum_tb,
        Cout => Cout_tb
        );

    stimulus_proc: process
    begin
        A_tb <= "0010";
        B_tb <= "0110";
        Cin_tb <= '0';
        wait for 10 ns;

        A_tb <= "0101";
        B_tb <= "1100";
        Cin_tb <= '0';
        wait for 10 ns;

        A_tb <= "1111";
        B_tb <= "0011";
        Cin_tb <= '1';
        wait for 10 ns;

        A_tb <= "1000";
        B_tb <= "0110";
        Cin_tb <= '1';
        wait for 10 ns;

        wait;
    end process stimulus_proc;

end tb_arch;
```

Το παραπάνω testbench εφαρμόζει ενδεικτικά στον παράλληλο αθροιστή 4 bit τέσσερα test cases, τα οποία όμως συνδυάζοντας όλες τις καταστάσεις των εισόδων επαρκούν για να επαληθεύσουν την αντίστοιχη λειτουργία των εξόδων. Συγκεκριμένα, πραγματοποιούνται οι προσθέσεις που φαίνονται παρακάτω:

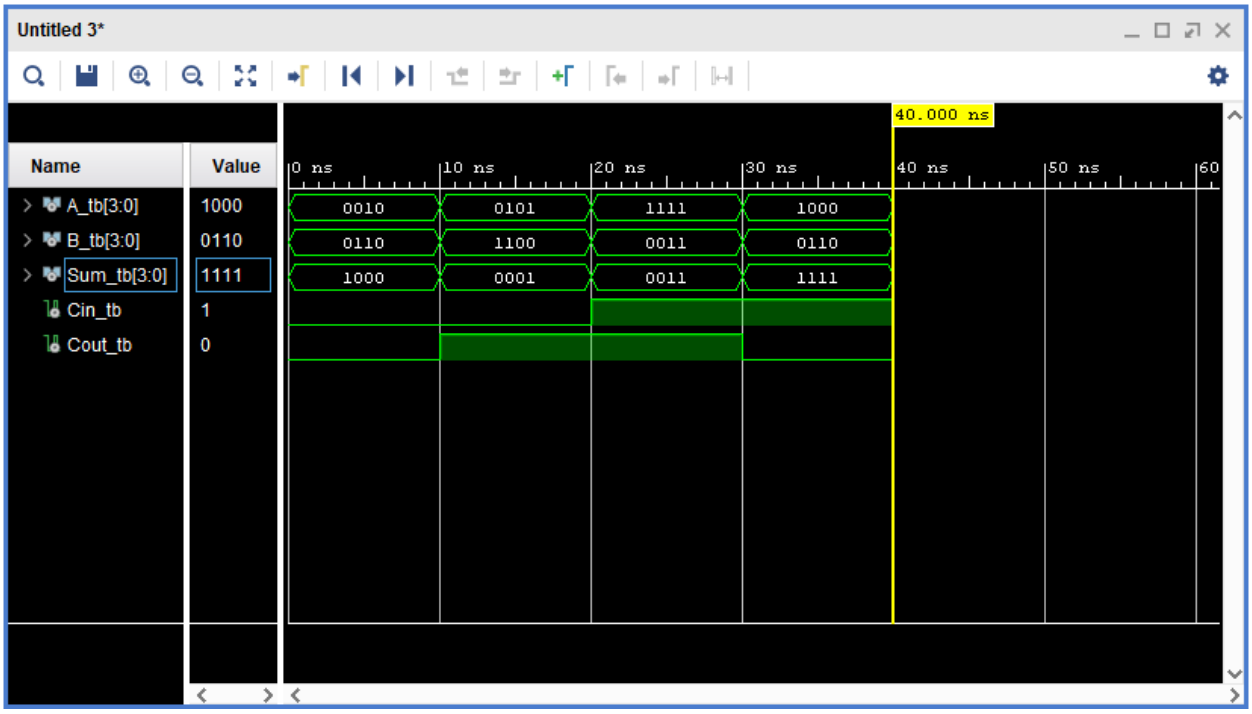
0010 + 0110 με Cin=0 → Sum=1000 με Cout=0

0101 + 1100 με Cin=0 → Sum=0001 με Cout=1

1111 + 0011 με Cin=1 → Sum=0011 με Cout=1

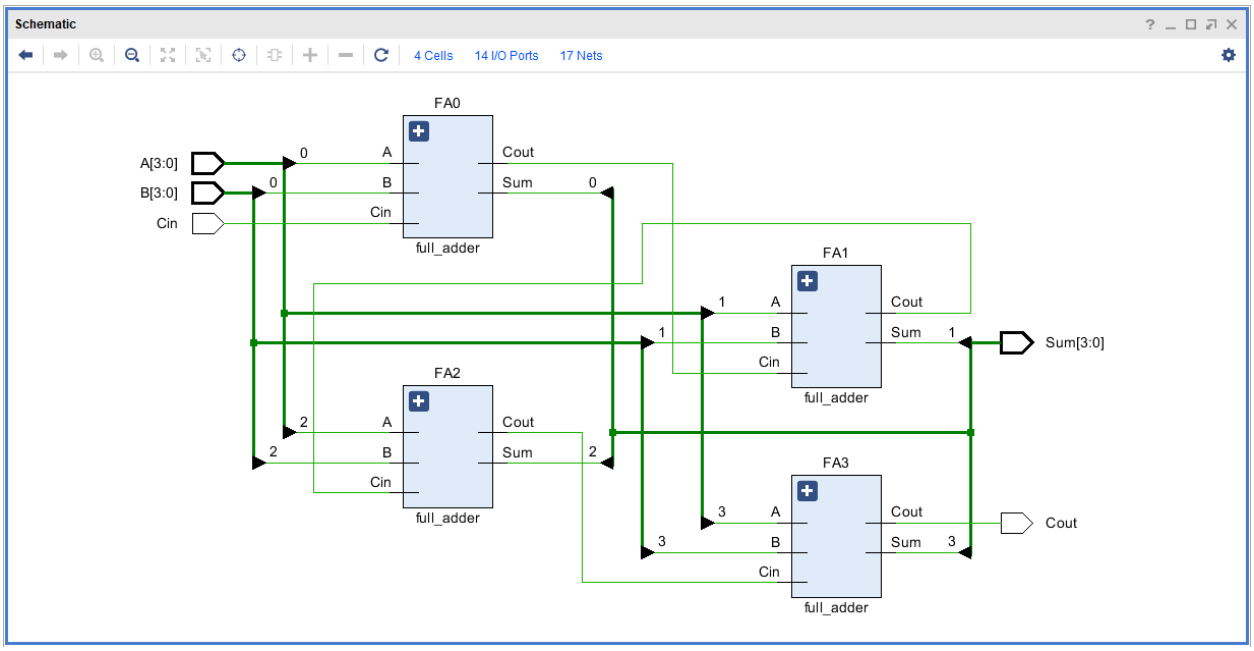
1000 + 0110 με Cin=1 → Sum=1111 με Cout=0

Η προσομοίωση του κυκλώματος του 4-bit Parallel Adder φαίνεται παρακάτω:



Παρατηρούμε ότι τα αποτελέσματα της προσομοίωσης για τις εξόδους Sum και Cout βρίσκονται σε πλήρη αντιστοιχία με τους θεωρητικούς υπολογισμούς που πραγματοποιήσαμε παραπάνω, επομένως το κύκλωμα του παράλληλου αθροιστή 4 bit λειτουργεί ορθά.

Το RTL του κυκλώματος του 4-bit Parallel Adder φαίνεται παρακάτω:



Το κρίσιμο μονοπάτι του 4-bit Parallel Adder ξεκινάει από την είσοδο B[0] και καταλήγει στην έξοδο Cout με συνολική καθυστέρηση 5.970 ns. Παρουσιάζει, όμως, ίδια συνολική καθυστέρηση με μονοπάτι που ξεκινάει από την είσοδο B[0] και καταλήγει στην έξοδο Sum[2].

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	4	5	3	B[0]	Cout	5.970	3.904	2.066
Path 2	∞	4	5	3	B[0]	Sum[2]	5.970	3.904	2.066
Path 3	∞	4	5	3	B[0]	Sum[3]	5.964	3.898	2.066
Path 4	∞	3	4	3	B[0]	Sum[0]	5.351	3.752	1.599
Path 5	∞	3	4	2	B[1]	Sum[1]	5.351	3.752	1.599

Ζητούμενο 4: BCD Full Adder – BCD FA

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity bcd is
    port ( A, B : in  std_logic_vector(3 downto 0);
           Cin : in std_logic;
           Sum : out  std_logic_vector(3 downto 0);
           Cout : out std_logic);
end bcd;

architecture structural of bcd is

    component four_bit_adder
        port ( A, B : in  std_logic_vector(3 downto 0);
               Cin : in std_logic;
               Sum : out  std_logic_vector(3 downto 0);
               Cout : out std_logic);
    end component;

    signal Sum1, B1 : std_logic_vector(3 downto 0);
    signal Carry1, Carry2, Carry3 : std_logic;

begin
    BinSum: four_bit_adder port map (
        A => A,
        B => B,
        Sum => Sum1,
        Cin => Cin,
        Cout => Carry1
    );

    Carry2 <= Carry1 or (Sum1(3) and Sum1(2)) or (Sum1(3) and Sum1(1));

    B1(0) <= '0';
    B1(1) <= Carry2;
    B1(2) <= Carry2;
    B1(3) <= '0';

    BCDSum: four_bit_adder port map (
        A => Sum1,
        B => B1,
        Sum => Sum,
        Cin => '0',
        Cout => Carry3
    );

    Cout <= Carry2;
end structural;
```

Ένα BCD ψηφίο χρησιμοποιεί 4 δυαδικά bit για την αναπαράστασή του και μπορεί να πάρει τιμές από 0 ("0000") έως 9 ("1001"), αφού μιλάμε για δεκαδικό σύστημα αναπαράστασης. Επομένως, η έξοδος ενός BCD αθροιστή με δύο εισόδους θα κυμαίνεται από το 0 μέχρι και το 18, εάν δεν λαμβάνουμε υπόψιν κάποιο κρατούμενο εισόδου. Αφού, όμως, εδώ η υλοποίηση πρόκειται για πλήρη αθροιστή BCD τότε έχουμε κρατούμενο εισόδου και η έξοδος τελικά θα πηγαίνει μέχρι το 19. Εάν το άθροισμα είναι μικρότερο ή ίσο του 9, τότε η τιμή του BCD αθροίσματος και του δυαδικού αθροίσματος θα είναι η ίδια, διαφορετικά θα διαφέρουν κατά 6 ("0110") και θα πρέπει να βρούμε ένα τρόπο να υπολογίζουμε το κρατούμενο εξόδου και να κάνουμε την μετατροπή από δυαδικό σε BCD.

Ο πίνακας αληθείας του BCD πλήρους αθροιστή φαίνεται παρακάτω:

Decimal	Binary Sum					BCD Sum				
	C'	S3'	S2'	S1'	S0'	C	S3	S2	S1	S0
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0
19	1	0	0	1	1	1	1	0	0	1

Η λογική που χρειάζεται για πετύχουμε την επιθυμητή μετατροπή είναι η εξής:
Cout = S3'S1' + S3'S2' + C' = 1

Ο παραπάνω κώδικας, λοιπόν, υλοποιεί έναν BCD πλήρη αθροιστή με Structural αρχιτεκτονική χρησιμοποιώντας δύο 4-bit Parallel Adder ως components και επιπλέον λογική για την μετατροπή και το κρατούμενο εξόδου. Ο πρώτος 4-bit PA πραγματοποιεί το δυαδικό άθροισμα των εισόδων A και B μαζί με το κρατούμενο εισόδου Cin. Έπειτα, υλοποιούμε την λογική που υπολογίσαμε παραπάνω για το κρατούμενο εξόδου Cout και την μετατροπή από δυαδικό σε BCD. Τέλος, ο δεύτερος 4-bit PA παίρνει το δυαδικό άθροισμα και με την προηγούμενη λογική προσθέτει 0 ή 6 αναλόγως για να γίνει η μετατροπή σε BDC άθροισμα (Sum).

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity bcd_tb is
end bcd_tb;

architecture tb_arch of bcd_tb is

    component bcd
        Port ( A, B : in  std_logic_vector(3 downto 0);
               Cin : in std_logic;
               Sum : out  std_logic_vector(3 downto 0);
               Cout : out std_logic);
    end component;

    signal A_tb, B_tb, Sum_tb : std_logic_vector(3 downto 0);
    signal Cin_tb, Cout_tb : std_logic;

begin

    DUT: bcd port map (
        A => A_tb,
        B => B_tb,
        Cin => Cin_tb,
        Sum => Sum_tb,
        Cout => Cout_tb
    );

    stimulus_process: process

    begin
        A_tb <= "0100";
        B_tb <= "0010";
        Cin_tb <= '0';
        wait for 10 ns;

        A_tb <= "1001";
        B_tb <= "0001";
        Cin_tb <= '1';
        wait for 10 ns;
```

```
41
42          A_tb <= "0110";
43          B_tb <= "0111";
44          Cin_tb <= '1';
45          wait for 10 ns;
46
47          A_tb <= "1000";
48          B_tb <= "0101";
49          Cin_tb <= '0';
50          wait for 10 ns;
51
52          wait;
53      end process;
54
55  end tb_arch;
```

Το παραπάνω testbench εφαρμόζει ενδεικτικά στον BCD πλήρη αθροιστή τέσσερα test cases, τα οποία όμως συνδυάζοντας όλες τις καταστάσεις των εισόδων επαρκούν για να επαληθεύσουν την αντίστοιχη λειτουργία των εξόδων. Συγκεκριμένα, πραγματοποιούνται οι προσθέσεις που φαίνονται παρακάτω:

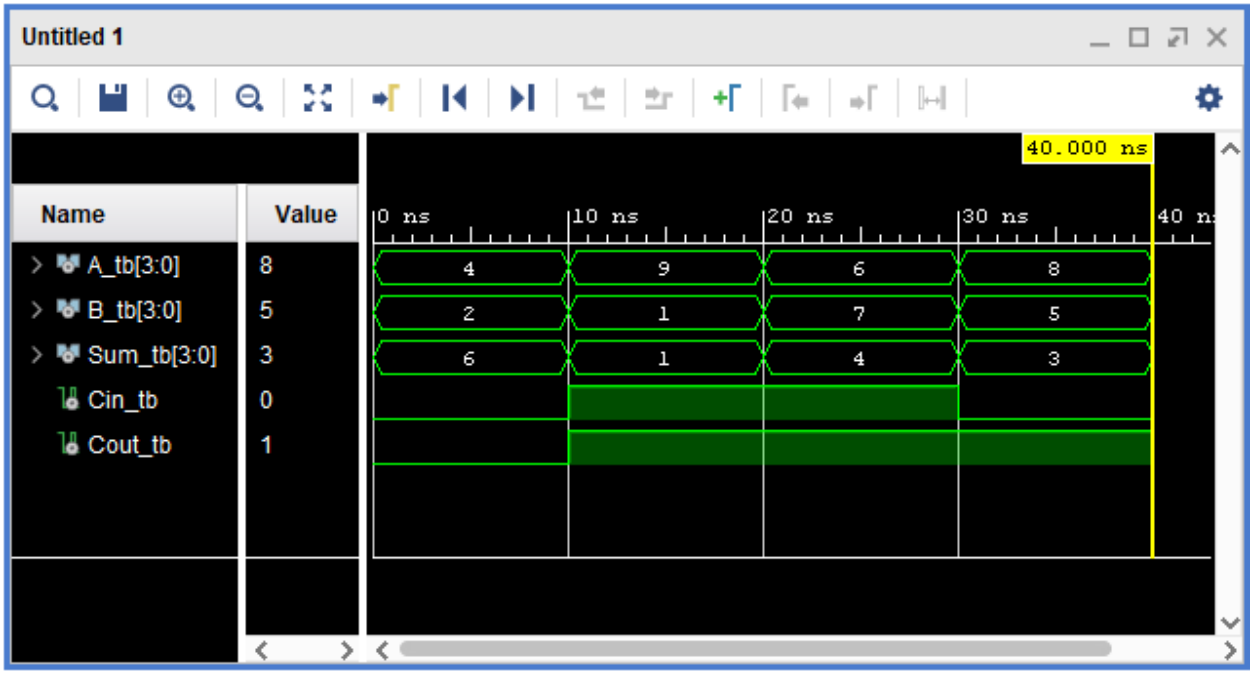
4 (0100) + 2 (0010) με Cin=0 → Sum=6 (0110) με Cout=0

9 (1001) + 1 (0001) με Cin=1 → Sum=1 (0001) με Cout=1

6 (0110) + 7 (0111) με Cin=1 → Sum=4 (0100) με Cout=1

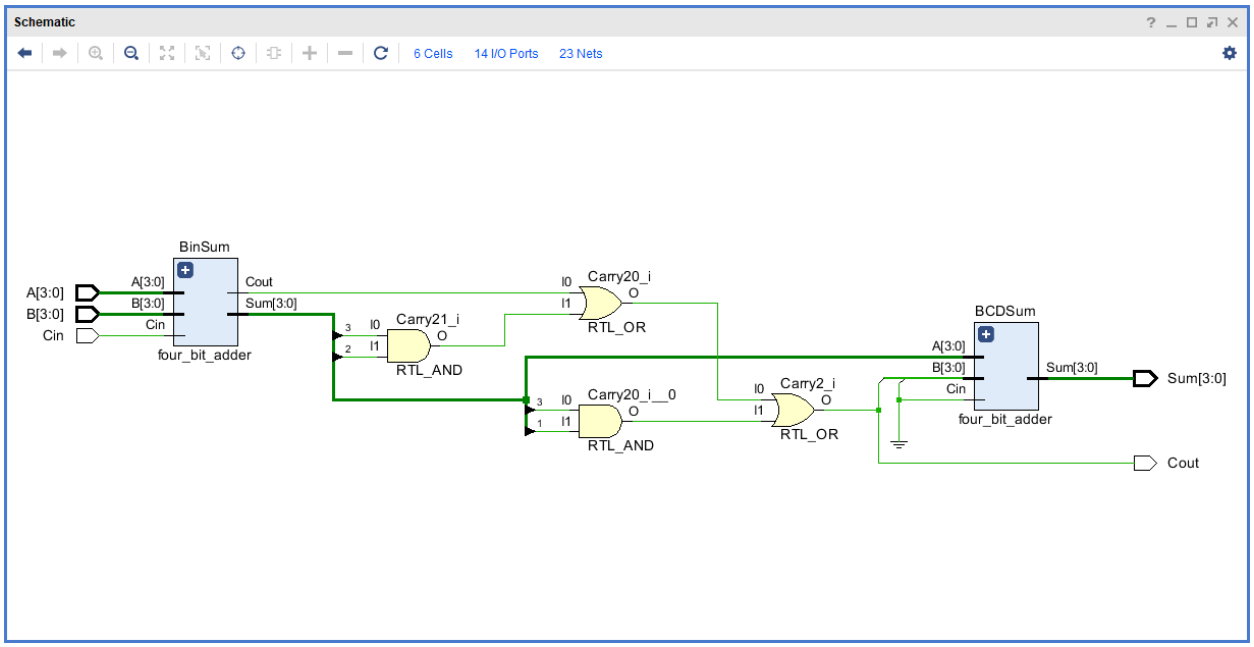
8 (1000) + 5 (0101) με Cin=0 → Sum=3 (0011) με Cout=1

Η προσομοίωση του κυκλώματος του BCD Full Adder φαίνεται παρακάτω:



Παρατηρούμε ότι τα αποτελέσματα της προσομοίωσης για τις εξόδους Sum και Cout βρίσκονται σε πλήρη αντιστοιχία με τους θεωρητικούς υπολογισμούς που πραγματοποιήσαμε παραπάνω, επομένως το κύκλωμα του BCD πλήρη αθροιστή λειτουργεί ορθά.

Το RTL του κυκλώματος του BCD Full Adder φαίνεται παρακάτω:



Το κρίσιμο μονοπάτι του BCD Full Adder ξεκινάει από την είσοδο B[0] και καταλήγει στην έξοδο Sum[3] με συνολική καθυστέρηση 5.976 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	4	5	4	B[0]	Sum[3]	5.976	3.904	2.072
Path 2	∞	4	5	4	B[1]	Cout	5.948	3.876	2.072
Path 3	∞	4	5	4	B[1]	Sum[1]	5.948	3.876	2.072
Path 4	∞	4	5	4	B[1]	Sum[2]	5.948	3.876	2.072
Path 5	∞	3	4	3	B[0]	Sum[0]	5.351	3.752	1.599

Ζητούμενο 5: 4-BCD Parallel Adder – 4-BCD PA

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity four_bit_bcd is
    port ( A, B : in  std_logic_vector(15 downto 0);
           Cin  : in  std_logic;
           Sum  : out std_logic_vector(15 downto 0);
           Cout : out std_logic);
end four_bit_bcd;

architecture structural of four_bit_bcd is

    component bcd is
        port ( A, B : in  std_logic_vector(3 downto 0);
               Cin : in std_logic;
               Sum : out  std_logic_vector(3 downto 0);
               Cout : out std_logic);
    end component;

    signal Carry : std_logic_vector(2 downto 0);

begin

    BCD1 : bcd port map (
        A => A(3 downto 0),
        B => B(3 downto 0),
        Cin => Cin,
        Sum => Sum(3 downto 0),
        Cout => Carry(0)
    );

    BCD2 : bcd port map (
        A => A(7 downto 4),
        B => B(7 downto 4),
        Cin => Carry(0),
        Sum => Sum(7 downto 4),
        Cout => Carry(1)
    );

    BCD3 : bcd port map (
        A => A(11 downto 8),
        B => B(11 downto 8),
        Cin => Carry(1),
        Sum => Sum(11 downto 8),
        Cout => Carry(2)
    );

    BCD4 : bcd port map (
        A => A(15 downto 12),
        B => B(15 downto 12),
        Cin => Carry(2),
        Sum => Sum(15 downto 12),
        Cout => Cout
    );

end structural;
```

Ο παραπάνω κώδικας υλοποιεί έναν παράλληλο BCD αθροιστή 4 ψηφίων με Structural αρχιτεκτονική χρησιμοποιώντας τέσσερις BCD Full Adder ως components. Κάθε ένας από τους αθροιστές χειρίζεται ένα από τα τέσσερα ψηφία της εξόδου Sum και με συνδεσμολογία cascade, όπου το κρατούμενο εξόδου ενός αθροιστή συνδέεται με τον κρατούμενο εισόδου του επόμενου, γίνεται η διάδοση του κρατούμενου. Προφανώς στον BCD Full Adder που χειρίζεται το LSB γίνεται και πρόσθεση του κρατούμενου εισόδου Cin του παράλληλου αθροιστή και, αντίστοιχα, ο BCD Full Adder που χειρίζεται το MSB δίνει το κρατούμενο εξόδου Cout.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity four_bit_bcd_tb is
5   end four_bit_bcd_tb;
6
7   architecture tb_arch of four_bit_bcd_tb is
8
9       component four_bit_bcd is
10          port ( A, B : in  std_logic_vector(15 downto 0);
11                 Cin  : in  std_logic;
12                 Sum  : out std_logic_vector(15 downto 0);
13                 Cout : out std_logic);
14      end component;
15
16      signal A_tb, B_tb, Sum_tb : std_logic_vector(15 downto 0);
17      signal Cin_tb, Cout_tb : std_logic;
18
19  begin
20
21      DUT: four_bit_bcd port map (
22          A => A_tb,
23          B => B_tb,
24          Cin => Cin_tb,
25          Sum => Sum_tb,
26          Cout => Cout_tb
27      );
28
29      stim_proc: process
30      begin
31          A_tb <= "0000000101110101"; --  175
32          B_tb <= "0000001101000111"; -- +347
33          Cin_tb <= '0';              -- 522 = "0000 0101 0010 0010"
34          wait for 10 ns;
35
```

```
36          A_tb <= "0110010100010011"; --  6513
37          B_tb <= "0101010000100110"; -- +5426
38          Cin_tb <= '0';              -- (Cout=1)1939 = "0001 1001 0011 1001"
39          wait for 10 ns;
40
41          A_tb <= "0000000000000110"; --  6
42          B_tb <= "0000000000000011"; -- +3
43          Cin_tb <= '1';              -- +1 (Cin)
44          wait for 10 ns;             -- 10="0000000000010000"
45
46          wait;
47      end process;
48
49  end tb_arch;
```

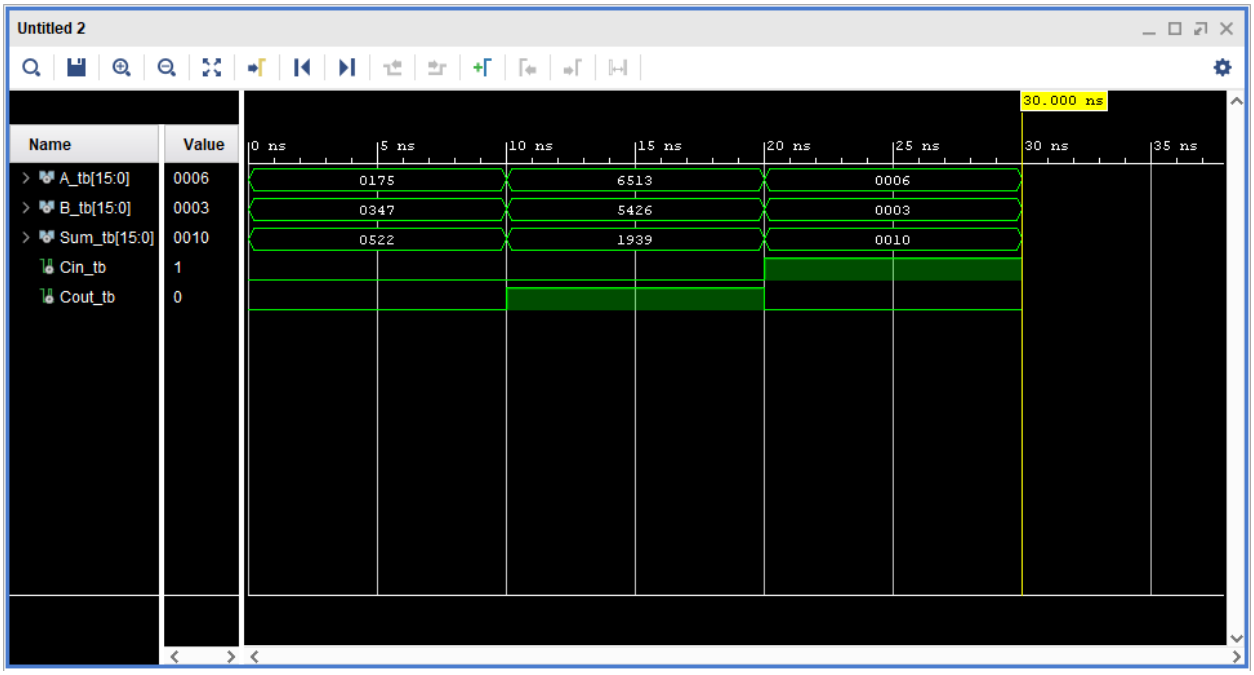
Το παραπάνω testbench εφαρμόζει ενδεικτικά στον παράλληλο BCD αθροιστή 4 ψηφίων τρία test cases, τα οποία όμως συνδυάζοντας όλες τις καταστάσεις των εισόδων επαρκούν για να επαληθεύσουν την αντίστοιχη λειτουργία των εξόδων. Συγκεκριμένα, πραγματοποιούνται οι προσθέσεις που φαίνονται παρακάτω:

175 + 347 με Cin=0 → Sum=522 με Cout=0

6513 + 5426 με Cin=0 → Sum=1939 με Cout=1

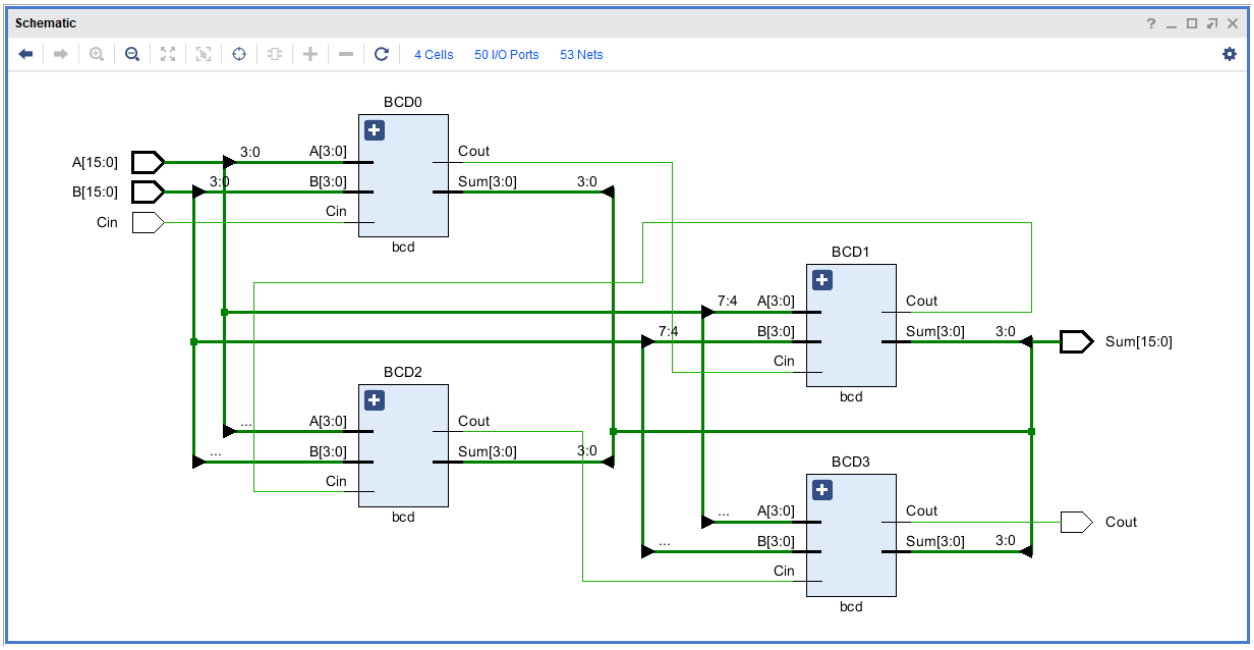
6 + 3 με Cin=1 → Sum=10 με Cout=0

Η προσομοίωση του κυκλώματος του 4-BCD Parallel Adder φαίνεται παρακάτω:



Παρατηρούμε ότι τα αποτελέσματα της προσομοίωσης για τις εξόδους Sum και Cout βρίσκονται σε πλήρη αντιστοιχία με τους θεωρητικούς υπολογισμούς που πραγματοποιήσαμε παραπάνω, επομένως το κύκλωμα του παράλληλου BCD αθροιστή 4 ψηφίων λειτουργεί ορθά.

Το RTL του κυκλώματος του 4-BCD Parallel Adder φαίνεται παρακάτω:



Το κρίσιμο μονοπάτι του 4-BCD Parallel Adder ξεκινάει από την είσοδο B[0] και καταλήγει στην έξοδο Sum[14] με συνολική καθυστέρηση 9.538 ns.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	10	11	5	B[0]	Sum[14]	9.538	4.642	4.896
Path 2	∞	10	11	5	B[0]	Cout	9.532	4.636	4.896
Path 3	∞	10	11	5	B[0]	Sum[13]	9.532	4.636	4.896
Path 4	∞	10	11	5	B[0]	Sum[15]	9.532	4.636	4.896
Path 5	∞	9	10	5	B[0]	Sum[12]	8.941	4.518	4.423
Path 6	∞	9	10	5	B[0]	Sum[9]	8.935	4.512	4.423
Path 7	∞	8	9	5	B[0]	Sum[10]	8.344	4.394	3.950
Path 8	∞	8	9	5	B[0]	Sum[11]	8.344	4.388	3.956
Path 9	∞	7	8	5	B[0]	Sum[5]	7.753	4.270	3.483
Path 10	∞	7	8	5	B[0]	Sum[8]	7.753	4.270	3.483